

SEGMENT 09 • THE INTEROP SEGMENT

Your existing framework. Sarvam underneath.

LangChain, LangGraph, CrewAI, n8n — one base-URL swap and a stock MCP server

Nothing new to learn • everything you already ship keeps shipping • four frameworks in one lab

Dr. Bhaveshkumar C. Dharmani • PhD (ICT), DA-IICT Gandhinagar • Founder, AI Vidhya4Sarvam

<https://www.aividhya.in/> • <https://www.aividhya4sarvam.in/> • bhavesh@aividhya.in • <https://www.linkedin.com/in/bhaveshdharmani/>

Why this segment exists at all

1

Half the room already ships on LangChain, LangGraph, CrewAI or n8n

Telling that team to rewrite onto a new SDK is a non-starter. The interesting question is whether their existing pipeline can point at Sarvam without changing shape.

2

The answer is yes, and the change fits in one line

Sarvam followed the OpenAI wire protocol precisely, and shipped an MCP server that speaks the stock protocol. Every framework built on either of those two contracts inherits Sarvam for free.

3

This is the segment that unblocks adoption in existing shops

It is also the segment that reframes 'should we use Indus?' as a build-vs-buy question rather than a religion. If your orchestrator is already good, you keep it — you just point it at a different endpoint.



You are not migrating to Sarvam. You are giving your existing stack a new endpoint. That is the whole segment in one sentence.

Two contracts, four frameworks, zero rewrites

1 The chat contract — OpenAI wire protocol

- Endpoint: POST /v1/chat/completions
- Auth: Authorization: Bearer <key>
- Request shape: messages[], tools[], stream, temperature...
- Response shape: choices[].message.content, tool_calls
- SSE for streaming — identical event framing
- Any framework that wraps OpenAI works by pointing base_url at api.sarvam.ai

2 The tools contract — Model Context Protocol

- sarvam-mcp: uvx sarvam-mcp, stdio transport, standard MCP
- 30 tools: 23 runtime (sarvam_tools_*) + 7 builder (sarvam_code_*)
- Runtime: STT, TTS, translate, transliterate, LID, LLM, vision, pronunciation, dub, localize
- Every MCP-aware framework picks it up: LangChain adapters, CrewAI tools, n8n MCP node
- One SARVAM_API_KEY env var, one config block
- No custom SDK bindings anywhere in your code



One protocol for chat, one protocol for tools. Everything downstream of those two protocols inherits Sarvam without knowing the vendor name.

The base-URL swap, four ways

Framework	The change	Model string
LangChain (Python)	ChatOpenAI(base_url="https://api.sarvam.ai/v1", api_key=..., model=...)	"sarvam-105b"
LangGraph	Uses your LangChain ChatOpenAI — same swap; StateGraph and create_agent unchanged	same as LangChain
CrewAI	LLM(model="openai/sarvam-105b", base_url=..., api_key=...) — base_url is what actually matters; without it BOTH forms hit real OpenAI	"openai/sarvam-105b"
n8n (no-code)	OpenAI credential → Base URL: https://api.sarvam.ai/v1 · then TYPE the model name in the node (dropdown is OpenAI-only)	"sarvam-105b" — typed manually
Vercel AI SDK	createOpenAI({ baseUrl: "https://api.sarvam.ai/v1", apiKey: ... })	"sarvam-105b"
Any HTTP tool (curl, n8n HTTP node, Zapier Webhook)	POST to https://api.sarvam.ai/v1/chat/completions with the OpenAI JSON body — no SDK at all	same



This is the slide you screenshot. Six frameworks, one swap each, one page. Everything else in this deck is a proof for one of these rows.



The four walkthroughs

LangChain · LangGraph · CrewAI · n8n — minimal, runnable, in that order

LangChain — ChatOpenAI is already a Sarvam client

PYTHON • pip install langchain langchain-openai

```
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage, SystemMessage

llm = ChatOpenAI(
    model = "sarvam-105b",           # sarvam-m/30b deprecated
    base_url = "https://api.sarvam.ai/v1", # <- the one line
    api_key = os.environ["SARVAM_API_KEY"],
    temperature= 0.2,
    max_tokens = 800,
)

reply = llm.invoke([
```

1

Tools work through the standard LangChain binding

`llm.bind_tools([...]).invoke(...)` — the same `@tool` decorator, the same schema, the same `tool_calls` response. Nothing framework-side changes.



reasoning_effort is now a first-class OpenAI parameter

It is a typed kwarg in `openai>=2.41` and a declared field on `ChatOpenAI`. Pass it DIRECTLY — `ChatOpenAI(..., reasoning_effort=None)`. Routing it through `model_kwargs` still works but raises a `UserWarning` telling you to stop.

LangChain + Sarvam MCP — 30 tools with zero glue

PYTHON • pip install langchain-mcp-adapters

```
from langchain_mcp_adapters.client import MultiServerMCPClient
from langchain.agents import create_agent # langgraph.prebuilt version deprecated

client = MultiServerMCPClient({
    "sarvam": {
        "command": "uvx",
        "args": ["sarvam-mcp"],
        "transport": "stdio",
        "env": {"SARVAM_API_KEY": os.environ["SARVAM_API_KEY"]},
    }
})

tools = await client.get_tools() # 30 tools, ready to bind
```

```
agent = create_agent(llm, tools=tools)
```

It **sarvam_sit_transcribe, sarvam_translate, sarvam_llm_complete** — all appear as ordinary LangChain tools

```
[{"role": "user", "content": "Transcribe ./data/hi_call.wav in Hindi"}]]})
```

The agent picks the right one from the prompt. You did not write a single wrapper. That is the value.



The MCP adapter is the reason your existing agent can suddenly transcribe Hindi audio, translate to Tamil and speak it back — with no new code in your project.

LangGraph — same LLM, graph you already know

PYTHON • pip install langgraph • reuse the ChatOpenAI llm from slide 6

```
from typing import TypedDict
from langgraph.graph import StateGraph, END

class S(TypedDict):
    query: str; language: str; answer: str

def detect(state):    # node 1 — call Sarvam LLM to classify
    r = llm.invoke([("system", "Return only the ISO code, e.g. hi-IN."),
                    ("user",   state["query"])]])
    return {"language": r.content.strip()}

def answer(state):    # node 2 — reply in the detected language
    r = llm.invoke([("system", f"Reply in {state['language']}. One sentence."),
                    ("user",   state["query"])]])
    return {"answer": r.content}
```



The graph is the value — the model is a swap

```
StateGraph(S):
    add_node("detect", detect)
    add_node("answer", answer)
    add_node("END", END)
    set_entry_point("detect"); g.add_edge("detect", "answer"); g.add_edge("answer", END)
```

You did not touch StateGraph, add_node, add_edge, checkpoints, interrupt() or human-in-the-loop. You changed the endpoint. Your LangGraph investment is intact.

```
app = g.compile()
print(app.invoke({"query": "मेरा खाना कब खुला था?"})["answer"])
```


CrewAI — the transport is chosen by Python TYPE

PYTHON • pip install crewai 'crewai-tools[mcp]' mcpadapt

```
from crewai import Agent, Task, Crew, LLM
from crewai_tools import MCPServerAdapter
from mcp import StdioServerParameters          # <- NOT a plain dict

llm = LLM(model      = "openai/sarvam-105b",    # prefix still the safe default
           base_url   = "https://api.sarvam.ai/v1",
           api_key    = os.environ["SARVAM_API_KEY"],
           max_tokens = 4000)                  # reasoning cannot be disabled here

server = StdioServerParameters(                # a dict here routes to SSE and
    command="uvx", args=["sarvam-mcp"],        # dies: sse_client() got an
    env={"SARVAM_API_KEY": os.environ["SARVAM_API_KEY"]}) # unexpected 'command'

with MCPServerAdapter(server) as tools:
    speech = Agent(role="Speech Analyst", llm=llm, tools=tools, goal="-",
                   backstory="You handle inbound support calls end-to-end.")
    crewai.LLM cannot disable reasoning — at all).kickoff_async()
```

!

call() strips every None-valued param before serialising, so reasoning_effort=None never reaches Sarvam and its default (reasoning on) applies. Your only lever is a generous max_tokens — and even that is not a guarantee.

n8n — no code, same trick, one credential

1

Create an OpenAI credential — override the Base URL

Credentials → OpenAI API → Base URL: `https://api.sarvam.ai/v1` · API Key: your Sarvam key. That single credential now works with every OpenAI node in n8n.

2

In the OpenAI Chat Model node, TYPE the model name

The model dropdown lists OpenAI's catalogue. Click the field, type `sarvam-105b`, hit enter. The node passes it straight through to Sarvam.

3

For the 13 non-chat APIs — install the MCP Client (Community) node

Point it at `command="uvx" args=["sarvam-mcp"]` with `SARVAM_API_KEY` in env. All 30 tools appear in the workflow palette. Chain them like any other node.

★

The escape hatch — the raw HTTP Request node

If a node does not exist or acts up, POST directly to `https://api.sarvam.ai/v1/chat/completions` with the OpenAI JSON body. It is the last-resort integration you never actually need.

★

n8n is the demo of the whole segment: not one line of code, and the workflow you already built now speaks Hindi. Show this to a non-engineer buyer.

B

The gotchas, and when to interop at all

The four things that will bite you, and the decision framework

The interop gotchas — every one found live

Where	Symptom	Fix
MCP tool result is a LIST of blocks	400: messages.N.tool.content — Input should be a valid string	MultiServerMCPClient(..., tool_interceptors=[...]) → flatten to a string
...and the interceptor covers only success	Failed tool calls still 400	Flatten the error path too
tts_speak returns AudioContent, not text	"(success, no text content)" → model retries, at real cost	Acknowledge audio/image/resource types explicitly
crewai MCPServerAdapter given a dict	sse_client() got an unexpected keyword 'command'	Pass mcp.StdioServerParameters(...) — type picks the transport
crewai_tools says 'mcp package missing'	Misleading — mcp IS installed	pip install mcpadapt (or 'crewai-tools[mcp]')
crewai.LLM drops reasoning_effort=None	content is None; 1–3 repeated ERROR lines	Cannot be disabled — raise max_tokens; expect sampling variance
langgraph.prebuilt.create_react_agent	DeprecationWarning (LangGraph v1.0)	from langchain.agents import create_agent
crew.kickoff() inside Jupyter	Conflicts with the running event loop	await crew.kickoff_async() — it is async, so await it
Stale kernel state after editing a cell	"I fixed it, why is it still broken"	Re-run the UPSTREAM cell; name fixed objects distinctly



Each of these cost real hours to find. Screenshot this slide — it is the cheapest artefact in the deck and the one most likely to save somebody a working day.

You did not rewire the house to change the bulb

Your house has a light socket. You changed the bulb from incandescent to LED. You did not rewire the house, replace the switch, or hire an electrician.

Why? Because everyone agreed on the shape of the socket a hundred years ago.

The OpenAI chat completions endpoint is that socket. MCP is that socket for tools. Sarvam ships a bulb that fits both.



This is why 'point your existing framework at Sarvam' is a slide and not a project. The socket is standard. You are only changing what is plugged into it.

Interop through a framework, or native on the SDK?

F Reach for interop when

- You already have a LangChain / LangGraph / CrewAI / n8n codebase in production
- The team's mental model is the framework, not the vendor SDK
- You want portability — swap Sarvam for another OpenAI-compatible model in one line
- You are prototyping fast and n8n gets you to a demo before lunch
- You need multi-vendor routing — Sarvam for Indic, OpenAI for frontier English, same graph
- The buyer wants to see a familiar framework in their stack

S Stay on the Sarvam SDK when

- You are consuming Sarvam-specific features — reasoning_effort, pronunciation dicts, batch STT modes
- You need the exact behaviour Sarvam's own docs describe, without a framework's abstraction leaking
- You are teaching or writing a cookbook — the SDK is what the docs reference
- You are chasing every millisecond of latency — one less abstraction layer helps
- Your production is a single service with one vendor and no orchestrator
- You want first-class support paths — the SDK is what the Sarvam team debugs first

★ **Neither answer is a religion. Most real projects use both — the SDK for the hot path, a framework for the orchestration around it. Choose per surface, not per stack.**

Take this home — your framework, your one-line change

If your team already ships...	...do this	Time to first call
LangChain	<code>ChatOpenAI(base_url=api.sarvam.ai/v1, api_key=..., model=...)</code>	3 minutes
LangGraph	Reuse LangChain ChatOpenAI in your nodes — no graph change	3 minutes
CrewAI	<code>LLM(model="openai/sarvam-105b", base_url=..., api_key=...)</code>	5 minutes
n8n	OpenAI credential → Base URL swap + type the model name	5 minutes
Vercel AI SDK	<code>createOpenAI({ baseUrl: api.sarvam.ai/v1, apiKey: ... })</code>	3 minutes
LlamaIndex	<code>OpenAI(api_base=api.sarvam.ai/v1, api_key=..., model=...)</code>	3 minutes
Nothing yet	Start on the Sarvam SDK — you get first-class support and docs	10 minutes
Non-Python stack	Any OpenAI SDK works · or POST /v1/chat/completions with curl	5 minutes



Twenty-eight minutes of total work across every mainstream orchestrator on the market. That is the size of the barrier to adopting Sarvam in an existing shop.



The lab you are about to run does all four in one notebook

LangChain, LangGraph, CrewAI and n8n — same task, four frameworks, ₹ cost printed for each. You leave the session with a running interop template.

“

The endpoint is standard.

The orchestrator is your choice.

Both of those are freedoms, not compromises.

— End of Segment 09. Lab 10 is the hands-on: four frameworks, one endpoint, one hour.